



VIGOR

Visual Integrated Governance & Overlap Registry

Projektrapport



11. FEBRUAR 2026

GRUPPE 11

Indhold

1	Indledning.....	5
1.1	Baggrund	5
1.2	Projektets kontekst (Bostedet “Slottet”)	5
1.3	Problemstilling	5
1.4	Problemformulering.....	6
1.5	1.5 Formål.....	6
1.6	1.6 Læsevejledning.....	6
2	Business case	6
2.1	Forretningsmæssig baggrund	6
2.2	Forretningsmæssig problemstilling	7
2.3	Interessenter	7
2.4	Systemets brugere	7
2.5	Scenarieoversigt.....	8
2.5.1	0-scenariet: Fortsætte med Excel og stramme op.....	8
2.5.2	1-scenariet: Udvikling af digitalt overlapsystem (valgt løsning)	8
2.5.3	2-scenariet: Manuel forbedring af arbejdsgange uden nyt system.....	9
2.5.4	Valgt scenarie	9
2.5.5	Implementering og opfølgning	11
3	Scope og Afgrænsning	11
3.1	In Scope	11
3.2	Out of Scope	12
3.3	Antagelser.....	12
3.4	Begrænsninger	12
4	Metode og Proces	12
4.1	Overordnet udviklingsmetode.....	12
4.2	Iterativ og feature-baseret tilgang.....	13
4.2.1	Iterationsstruktur	13
4.3	Kombination af Scrum og XP-principper	13
4.4	Artefaktstrategi (levende diagrammer).....	13

4.5	Versionsstyring og commit-strategi	13
4.6	Definition of Done.....	14
5	Forretningsforståelse og Krav	15
5.1	Interessenter.....	15
5.2	Aktører.....	15
5.2.1	Personale.....	15
5.2.2	Vagtansvarlig.....	15
5.2.3	Leder	15
5.2.4	Superbruger	15
5.3	Overordnet forretningsproces (vagtskifte).....	16
5.4	Use case Backlog	16
5.4.1	UC01 – Log ind	16
5.4.2	UC02 – Se borgere på afdeling	16
5.4.3	UC03 – Opdater borgerstatus / risikoprofil	16
5.4.4	UC04 – Registrér fast medicin	16
5.4.5	UC05 – Registrér PN medicin	17
5.4.6	UC06 – Se historik / audit-log.....	17
5.4.7	UC07 – Vælg vagt	17
5.4.8	UC08 – Se overlap.....	17
5.4.9	UC09 – Opret note	17
5.4.10	UC10 – Opret og afslut opgave	17
5.4.11	UC11 – Tildel personale til borger.....	18
5.4.12	UC12 – Tildel telefon til personale	18
5.4.13	UC13 – Administrér brugere og roller	18
5.4.14	UC14 – Public anonym oversigtsskærm	18
5.4.15	UC15 – Statistik og rapporter	18
5.5	Use Case Prioriteringstabel	18
5.6	Use Case Diagram	20
5.7	Brief Use Cases	21
6	Domænemodel og Analyse	21

6.1	Domænemodel (Analysemodel)	21
6.2	Begrebsafklaring.....	22
6.3	Analyseovervejelser	22
7	Systemsekvens og Interaktionsanalyse	22
7.1	System Sequence Diagram – Medicin.....	22
7.2	System Sequence Diagram – Vagtskifte	22
7.3	Analyse af systemgrænse	22
8	Design og Arkitektur	23
8.1	Overordnet arkitektur	23
8.2	Valg af teknologi (.NET MAUI Blazor Hybrid)	24
8.3	MVC-struktur.....	24
8.4	Dependency Injection	24
8.5	SOLID, GRASP, KISS, DRY	25
9	Designmodeller	25
9.1	Design Class Diagram (DCD)	25
9.2	SSD – UC01 – Login	25
9.3	Sekvensdiagram – UC01 – Login.....	26
9.4	Ansvarsfordeling i klasser	26
10	Datamodel	26
10.1	ER-diagram	26
10.2	Entiteter og relationer	26
10.3	Historik- og audit-strategi	26
11	Brugergrænseflade.....	27
11.1	Designprincipper	27
11.2	Wireframe – Public View	27
11.3	Wireframe – Login	27
11.4	Wireframe – Overlap screen	27
11.5	Brugervenlighed for IT-ukyndige	27
12	Medicinhåndtering	27
12.1	Aktivitetsdiagram – Fast medicin.....	27

12.2	PN medicin-flow	27
12.3	Validering og logning	27
13	Ikke-funktionelle Krav (Supplementary Specification)	27
13.1	Brugervenlighed	27
13.2	Sikkerhed	27
13.3	Performance	28
13.4	Tilgængelighed	28
13.5	Vedligeholdbarhed	28
14	Risikovurdering og GDPR	28
14.1	GDPR-kontekst	28
14.2	Risikotabel	28
14.3	Anonymisering vs. lukket system	28
14.4	Rollebaseret adgang	28
14.5	Audit-log og sporbarhed	28
14.6	Argumentation for løsning	28
15	Deployment og Infrastruktur	28
15.1	Deployment diagram	29
15.2	Lukket installationsscenario	29
15.3	Ekstern anonym adgang	29
15.4	Driftsovervejelser	29
16	Implementering	29
16.1	Projektstruktur	29
16.2	Mappestruktur og namespaces	30
16.3	Centrale services	30
16.4	Teststrategi	30
16.5	Eksempler på kode	30
17	Test og Kvalitet	30
17.1	Unit tests	30
17.2	Integrationstest	32
17.3	Manuel test	32

17.4	Kvalitetssikring	32
18	Refleksion	33
18.1	Metoderefleksion.....	33
18.2	Samarbejde og iterationer	33
18.3	Hvad fungerede godt.....	33
18.4	Hvad kunne forbedres	33
19	Konklusion	33
19.1	Opfyldelse af problemformulering	33
19.2	Fremtidige udvidelser	33
20	Referencer.....	33
21	Bilag.....	34
21.1	Ekstra diagrammer.....	34
21.2	Ekstra use cases.....	34
21.3	Screenshots.....	36
21.4	GitHub link.....	36

1 Indledning

1.1 Baggrund

1.2 Projektets kontekst (Bostedet “Slottet”)

1.3 Problemstilling

Bostedet “Slottet” bruger i dag et Excel-ark til at håndtere overlap mellem vagter. Det fungerer i praksis, men det er ikke lavet til formålet. Der bliver skrevet noter, registreret medicin og opdateret borgerstatus, men der er ingen rigtig struktur, ingen sporbarhed og ingen klar rollefordeling i systemet.

Der er især to udfordringer.

For det første mangler der overblik og historik. Det er ikke tydeligt, hvem der har ændret hvad, og hvornår det er gjort. Det kan være problematisk, især når det handler om medicin og risikovurderinger, hvor det er vigtigt at kunne dokumentere ændringer.

For det andet er der usikkerhed omkring GDPR. Excel-arket indeholder personfølsomme oplysninger, og regionen har gjort det klart, at navne og identificerbare oplysninger kun må vises i et lukket og godkendt system. Samtidig hænger der en skærm fremme 24 timer i døgnet, hvilket betyder, at information ikke bare kan vises frit.

Derudover sker overlap flere gange dagligt, og alle borgere skal gennemgås systematisk. Når værktøjet ikke er lavet til opgaven, øger det risikoen for fejl eller manglende overlevering.

Der er derfor behov for et system, der er lavet specifikt til formålet:

Et system der understøtter arbejdsgangen ved vagtskifte, sikrer korrekt håndtering af persondata, giver sporbarhed og samtidig er så simpelt, at alle medarbejdere kan bruge det – også dem der ikke er stærke i IT.

1.4 Problemformulering

Hvordan kan der udvikles et digitalt overlapsystem, som erstatter den nuværende Excel-løsning, og som samtidig:

- understøtter arbejdsgangen ved vagtskifte på en struktureret måde
- sikrer korrekt håndtering af medicin (både fast og PN)
- giver sporbarhed og historik over ændringer
- overholder GDPR, herunder krav om anonymisering og rollebaseret adgang
- er så brugervenligt, at selv ikke IT kyndige kan tage fejl af brugen

1.5 1.5 Formål

1.6 1.6 Læsevejledning

2 Business case

2.1 Forretningsmæssig baggrund

Bostedet "Slottet" anvender i dag et Excel-ark til at håndtere overlap mellem vagter, registrering af medicin og opdatering af borgernes risikoprofil. Systemet fungerer som et fælles arbejdsredskab for personalet og bruges flere gange dagligt i forbindelse med vagtskifte (ca. kl. 07, 15 og 23).

Excel-løsningen er ikke udviklet til formålet og mangler funktioner som rollebaseret adgang, sporbarhed og struktureret historik. Samtidig indeholder arket personfølsomme oplysninger, hvilket skaber usikkerhed i forhold til GDPR – især fordi informationer vises på en skærm, der er synlig 24 timer i døgnet.

Der er derfor behov for en løsning, der er udviklet specifikt til bostedets arbejdsgange og som understøtter både drift, dokumentation og lovgivning.

2.2 Forretningsmæssig problemstilling

Den nuværende arbejdsgang medfører følgende problemer:

- Der mangler sporbarhed: Det er uklart, hvem der har foretaget ændringer og hvornår.
- Medicinregistrering sker uden systematisk logning.
- Risikoprofiler skal opdateres mindst tre gange dagligt, men der er ingen automatisk understøttelse af dette krav.
- Personfølsomme oplysninger vises potentielt i et miljø, der ikke er GDPR-mæssigt godkendt.
- Overlap ved vagtskifte er afhængigt af manuelle rutiner og individuel disciplin.

Disse forhold øger risikoen for fejl, manglende dokumentation og usikkerhed omkring databeskyttelse.

2.3 Interessenter

Ledelsen på bostedet, som er ansvarlig for drift og overholdelse af lovgivning.

Personale og vagtansvarlige, som anvender systemet dagligt i forbindelse med vagtskifte og medicinbehandling.

Regionen, som stiller krav til databeskyttelse og dokumentation.

Borgere, hvis oplysninger behandles i systemet.

2.4 Systemets brugere

Personale

Vagtansvarlig

2.5 Scenarieoversigt

2.5.1 0-scenariet: Fortsætte med Excel og stramme op

Fordele

- Ingen investering i nyt system
- Ingen større ændring af arbejdsgange

Ulemper

- Fortsat manglende sporbarhed
 - Usikker GDPR-overholdelse
 - Risiko for fejl ved medicinregistrering
 - Vanskeligt at skalere eller dokumentere over for regionen
-

2.5.2 1-scenariet: Udvikling af digitalt overlapsystem (valgt løsning)

Fordele

- Rollebaseret adgang
- Automatisk historik og audit-log
- Anonym visning på public skærm
- Struktur ved vagtskifte
- Bedre dokumentation og GDPR-argumentation

Ulemper

- Udviklingstid
- Kræver ændring af arbejdsgange
- Kræver oplæring af personale

2.5.3 2-scenariet: Manuel forbedring af arbejdsgange uden nyt system

Fordele

- Ingen IT-investering

Ulemper

- Kræver høj disciplin
- Ingen teknisk sikring af sporbarhed
- Vanskeligt at dokumentere compliance

2.5.4 Valgt scenarie

Det foreslås at udvikle et digitalt overlapsystem, som understøtter arbejdsgange ved vagtskifte, medicin håndtering og risikostyring.

2.5.4.1 Formål

Formålet med overlapsystemet er:

- at skabe struktur og overblik ved vagtskifte
- at sikre korrekt og dokumenteret medicin håndtering
- at sikre sporbarhed og historik
- at overholde GDPR via rollebaseret adgang og anonym visning
- at gøre arbejdsgangen enklere og mere robust

2.5.4.2 Forretningsmæssig løsningsbeskrivelse

Den nuværende Excel-løsning erstattes af et system, hvor:

- Borgerstatus registreres struktureret
- Medicin registreres med tidsstempel og bruger-ID
- Risikoprofiler kan opdateres og logges
- Public skærm viser anonymiserede oplysninger
- Historik gemmes som append-only log

Løsningen implementeres internt på bostedet og anvendes af personale og ledelse.

2.5.4.3 *It-mæssig løsningsbeskrivelse*

Systemet udvikles som en klient-server løsning baseret på .NET-plattformen. Brugergrænsefladen implementeres som en .NET MAUI Blazor Hybrid-applikation, som fungerer som klient og håndterer præsentation, navigation og brugerinteraktion.

Backend-delen implementeres som et ASP.NET Core Web API. API'et håndterer systemets forretningslogik, dataadgang og autentificering. Klienten kommunikerer med serveren via HTTP-baserede API-kald, hvor data sendes og modtages i struktureret form.

Autentificering og brugerhåndtering håndteres ved hjælp af ASP.NET Identity. Ved login validerer serveren brugerens oplysninger og tildeler den relevante rollebaserede adgang. Efter succesfuld autentificering udstedes et JSON Web Token (JWT), som anvendes til efterfølgende autoriserede API-kald mellem klient og server.

Arkitekturen understøtter en tydelig adskillelse mellem præsentationslag, forretningslogik og dataadgang. Denne opdeling reducerer koblingen mellem systemets komponenter og gør løsningen mere vedligeholdelsesvenlig og udvidelsesvenlig. Samtidig gør klient-server strukturen det muligt senere at udvide systemet med yderligere klienter eller integrationsmuligheder uden større ændringer i backend-systemet.

2.5.4.4 *Økonomiske effekter*

Der er primært udviklingsomkostninger i form af arbejdstid.

Der forventes ikke væsentlige hardwareinvesteringer, da systemet kan køre på eksisterende udstyr.

På længere sigt kan systemet reducere tidsforbrug ved vagtskifte og mindske risikoen for fejl.

2.5.4.5 *Gevinster*

Interne forbedringer

- Bedre struktur
- Mindre risiko for fejl
- Tydelig ansvarsplacering
- Dokumentation ved tilsyn

Fleksibilitet

Systemet kan senere udvides med yderligere funktionalitet.

2.5.4.6 *Risici*

- Modstand mod nyt system
- IT-ukyndigt personale
- Forkert håndtering af GDPR

Risici håndteres gennem simpelt design, tydelig rolleopdeling og løbende test.

2.5.5 Implementering og opfølgning

2.5.5.1 *Implementeringsstrategi*

Systemet implementeres gradvist.

I en kort periode kan Excel og det nye system køre parallelt for at sikre korrekt overgang.

2.5.5.2 *Gevinstrealisering – KPI'er*

Eksempler på KPI'er:

- Andel af medicinregistreringer med korrekt logning
- Antal manglende risikoprofil-opdateringer
- Tidsforbrug ved vagtskifte
- Antal dokumentationsafvigelser ved tilsyn

3 Scope og Afgrænsning

3.1 In Scope

Projektet omfatter udvikling af et digitalt overlapsystem, der skal:

- understøtte vagtskifte mellem personalegrupper
- give overblik over borgere pr. afdeling
- vise og opdatere risikoprofil (trafiklys)
- registrere fast medicin
- registrere PN medicin med tidspunkt når givet
- give mulighed for at skrive noter og opgaver
- understøtte rollebaseret adgang (personale, vagtansvarlig, leder, superbruger)

- vise anonymiseret information på public skærm
- gemme historik og audit-log over ændringer

3.2 Out of Scope

Projektet omfatter ikke:

- integration med eksterne medicinsystemer
- integration med regionale systemer
- mobilapp-version
- automatisk notifikation via SMS eller mail
- avanceret statistik eller rapportmodul
- integration med eksterne API'er

3.3 Antagelser

3.4 Begrænsninger

4 Metode og Proces

(Den her bliver vigtig til 08-05)

4.1 Overordnet udviklingsmetode

Udviklingen af VIGOR følger en iterativ og inkrementel udviklingsmetode inspireret af agile principper. Projektet gennemføres individuelt, men organiseres efter en arbejdsform, der kombinerer elementer fra Scrum og Extreme Programming (XP).

Arbejdet struktureres i korte iterationer, hvor hver iteration tager udgangspunkt i én eller få prioriterede use cases fra systemets backlog. Formålet er at levere funktionel værdi løbende og samtidig sikre, at analyse, design og implementering udvikles parallelt og kan justeres undervejs.

Scrum-inspirationen ses især i anvendelsen af backlog, prioriterede use cases og en iterativ fremdrift, hvor funktionalitet implementeres trinvis. XP-principper anvendes i form af små, hyppige ændringer, løbende refactoring, fokus på simpel arkitektur og kontinuerlig integration gennem hyppige commits.

Hver iteration gennemføres som en mini-udviklingscyklus bestående af analyse, design, implementering og test. De valgte use cases fungerer som styrende element for både designartefakter og implementering. Når en use case implementeres, opdateres relevante diagrammer og dokumentation for at bevare sporbarhed mellem krav, design og kode.

Versionsstyring spiller en central rolle i processen. Arbejdet organiseres gennem Git og GitHub, hvor ændringer løbende commits og pushes til repository'et. Den iterative commit-strategi sikrer, at udviklingshistorikken er dokumenteret, og at systemet løbende stabiliseres gennem udviklingsforløbet.

Denne tilgang gør det muligt at arbejde struktureret med udviklingen samtidig med, at systemet gradvist forbedres og udvides gennem projektets iterationer.

4.2 Iterativ og feature-baseret tilgang

4.2.1 Iterationsstruktur

Udviklingen af VIGOR er organiseret i korte, afgrænsede iterationer med fokus på inkrementel fremdrift og løbende stabilisering af systemet. Hver iteration tager udgangspunkt i én eller få prioriterede use cases fra backloggen og gennemføres som en komplet mini-udviklingscyklus.

En iteration starter med afklaring af scope og konkretisering af den valgte funktionalitet. Herefter vurderes eventuelle konsekvenser for domænemodel, arkitektur og diagrammer. Implementeringen gennemføres i små, kontrollerede skridt, hvor der arbejdes med hyppige commits og løbende build-verificering.

Ved afslutning af iterationen verificeres det, at Definition of Done er opfyldt. Dette indebærer, at systemet bygger uden warnings eller errors, at funktionaliteten er testet manuelt, samt at relevant dokumentation og diagrammer er opdateret ved behov. Iterationen afsluttes med en stabil version i main-grenen.

Denne struktur sikrer sporbarhed mellem analyse, design og implementering og understøtter den iterative tilgang beskrevet i metodeafsnittet.

4.3 Kombination af Scrum og XP-principper

4.4 Artefaktstrategi (levende diagrammer)

4.5 Versionsstyring og commit-strategi

Projektet VIGOR versionsstyres ved anvendelse af Git og GitHub. Formålet med versionsstyringen er at sikre sporbarhed, transparens og mulighed for at genskabe tidligere stabile versioner af systemet gennem hele udviklingsforløbet. Versionsstyringen understøtter den iterative arbejdsform og sikrer, at hver iteration afsluttes i en dokumenteret og stabil tilstand.

Repository'et er struktureret med en hovedgren (main), som altid indeholder en stabil og kørende version af systemet. Udviklingsarbejde udføres i en separat udviklingsgren (dev), hvor nye funktioner og ændringer implementeres og testes, før de eventuelt merges til main. Denne struktur sikrer, at hovedgrenen til enhver tid kan anvendes til demonstration, evaluering eller videreudvikling uden risiko for ustabil kode.

Der arbejdes iterativt, og hver funktionel ændring afsluttes med en commit og push til GitHub. Commit-beskeder formuleres beskrivende og konsekvent for at sikre historisk overblik over udviklingen. Der anvendes en enkel og semantisk inspireret commit-struktur, eksempelvis:

chore: tekniske ændringer uden funktionel påvirkning

feat: tilføjelse af ny funktionalitet

fix: rettelse af fejl

Denne struktur bidrager til klar historik og øget læsbarhed i repository'et.

Da projektet udvikles individuelt, anvendes der ikke formelle Pull Requests eller peer review-processer. I stedet foretages kodegennemgang løbende af udvikleren selv før merge til main-grenen. Inden merge sikres det, at løsningen bygger uden warnings eller errors, og at funktionaliteten er testet manuelt i den aktuelle kontekst. Main-grenen indeholder således altid en stabil og dokumenteret version af systemet.

Ved afslutning af større milepæle oprettes tags i Git, så specifikke stabile versioner kan identificeres og genskabes. Iteration 0 er eksempelvis markeret som en baseline-version, der repræsenterer det tekniske fundament med korrekt projektstruktur, centraliseret Dependency Injection og et clean build.

Samlet set understøtter versionsstyringsstrategien både teknisk kvalitet og dokumentationskravene i semesterprojektet.

4.6 Definition of Done

For at sikre ensartet kvalitet og fremdrift er der fastlagt en fælles Definition of Done (DoD). En opgave betragtes først som afsluttet, når følgende kriterier er opfyldt.

Først og fremmest skal løsningen bygge uden warnings eller errors. Et clean build betragtes som et minimumskrav, da teknisk gæld ikke må videreføres mellem iterationer.

Derudover skal funktionaliteten være testet, som minimum manuelt, for at verificere at den opfører sig som forventet i den aktuelle kontekst. Hvis ændringen påvirker domænemodellen eller arkitekturen, skal relevante diagrammer og dokumentation opdateres, så sporbarheden mellem analyse, design og implementering bevares.

Endelig skal ændringen være committed og pushed til GitHub med en beskrivende commit-besked. Opgaven markeres herefter som afsluttet i Trello og flyttes til “Done”-kolonnen.

Definition of Done fungerer som en kvalitetssikringsmekanisme og understøtter projektets iterative arbejdsform. Den bidrager til at sikre, at hver iteration afsluttes i en stabil og dokumenteret tilstand.

5 Forretningsforståelse og Krav

5.1 Interessenter

- Bostedets personale
- Vagtansvarlige
- Ledelse
- Regionen (indirekte via GDPR-krav)

5.2 Aktører

5.2.1 Personale

Kan registrere medicin, opdatere borgerstatus og skrive noter.

5.2.2 Vagtansvarlig

Kan ændre risikoprofil og fordele opgaver.

5.2.3 Leder

Kan se historik og statistik.

5.2.4 Superbruger

Kan administrere roller og systemopsætning.

5.3 Overordnet forretningsproces (vagtskifte)

5.4 Use case Backlog

Nedenstående use cases udgør systemets samlede backlog. De er prioriteret ud fra forretningsværdi, sikkerhed og lovmæssige krav (herunder medicin håndtering og GDPR). Hver use case implementeres i en selvstændig iteration.

5.4.1 UC01 – Log ind

Primær aktør: Personale

Beskrivelse: Systemet skal give autoriseret personale adgang via brugernavn og adgangskode og tildele korrekt rollebaseret adgang. Login håndteres via systemets Web API, hvor brugeroplysninger valideres gennem ASP.NET Identity. Ved succesfuld autentificering udstedes et JWT-token, som anvendes til efterfølgende autoriserede forespørgsler og til at bestemme brugerens rollebaserede navigation i systemet.

Prioritet: MUST

Iteration: 1

5.4.2 UC02 – Se borgere på afdeling

Primær aktør: Personale

Beskrivelse: Systemet skal vise en liste over borgere på den valgte afdeling med relevante statusoplysninger.

Prioritet: MUST

Iteration: 2

5.4.3 UC03 – Opdatér borgerstatus / risikoprofil

Primær aktør: Personale

Beskrivelse: Systemet skal give mulighed for at opdatere borgerens aktuelle status og risikoprofil i overensstemmelse med gældende krav.

Prioritet: MUST

Iteration: 3

5.4.4 UC04 – Registrér fast medicin

Primær aktør: Personale

Beskrivelse: Systemet skal registrere, at planlagt (fast) medicin er udleveret til borgeren på et givent tidspunkt.

Prioritet: MUST

Iteration: 4

5.4.5 UC05 – Registrér PN medicin

Primær aktør: Personale

Beskrivelse: Systemet skal registrere udlevering af PN-medicin (ved behov), herunder tidspunkt for administration.

Prioritet: MUST

Iteration: 5

5.4.6 UC06 – Se historik / audit-log

Primær aktør: Personale / Leder

Beskrivelse: Systemet skal give adgang til historik over ændringer, medicinregistreringer og statusopdateringer for dokumentation og sporbarhed.

Prioritet: MUST

Iteration: 6

5.4.7 UC07 – Vælg vagt

Primær aktør: Personale

Beskrivelse: Systemet skal give mulighed for at vælge aktiv vagt, så handlinger registreres korrekt i forhold til tidspunkt og ansvar.

Prioritet: SHOULD

Iteration: 7

5.4.8 UC08 – Se overlap

Primær aktør: Personale

Beskrivelse: Systemet skal understøtte visning af relevant information ved vagtskifte for at sikre korrekt overlevering.

Prioritet: SHOULD

Iteration: 8

5.4.9 UC09 – Opret note

Primær aktør: Personale

Beskrivelse: Systemet skal give mulighed for at oprette faglige noter knyttet til en borger.

Prioritet: SHOULD

Iteration: 9

5.4.10 UC10 – Opret og afslut opgave

Primær aktør: Personale

Beskrivelse: Systemet skal give mulighed for at oprette opgaver og markere dem som udført.

Prioritet: SHOULD

Iteration: 10

5.4.11 UC11 – Tildel personale til borger

Primær aktør: Vagtansvarlig

Beskrivelse: Systemet skal give vagtansvarlig mulighed for at fordele personale til specifikke borgere.

Prioritet: SHOULD

Iteration: 11

5.4.12 UC12 – Tildel telefon til personale

Primær aktør: Vagtansvarlig

Beskrivelse: Systemet skal give mulighed for at registrere og fordele arbejdstelefoner til personale under vagten.

Prioritet: COULD

Iteration: 12

5.4.13 UC13 – Administrér brugere og roller

Primær aktør: Leder

Beskrivelse: Systemet skal give mulighed for at oprette, deaktivere og tildele roller til brugere.

Prioritet: SHOULD

Iteration: 13

5.4.14 UC14 – Public anonym oversigtsskærm

Primær aktør: Offentlig visning (uden login)

Beskrivelse: Systemet skal kunne vise en anonymiseret oversigtsskærm uden personhenførbare oplysninger, egnet til 24/7 visning i fællesområder.

Prioritet: SHOULD

Iteration: 14

5.4.15 UC15 – Statistik og rapporter

Primær aktør: Leder

Beskrivelse: Systemet skal give mulighed for at generere statistisk overblik og rapporter over aktivitet i systemet.

Prioritet: COULD

Iteration: 15

5.5 Use Case Prioriteringstabel

Jeg har prioriteret use cases ud fra, hvilken funktionalitet der skaber størst værdi og sikkerhed for bostedet først. Derfor har jeg placeret medicin håndtering, risikoprofil og dokumentation

øverst, da fejl her kan have direkte konsekvenser for borgernes sikkerhed og for overholdelse af GDPR-krav.

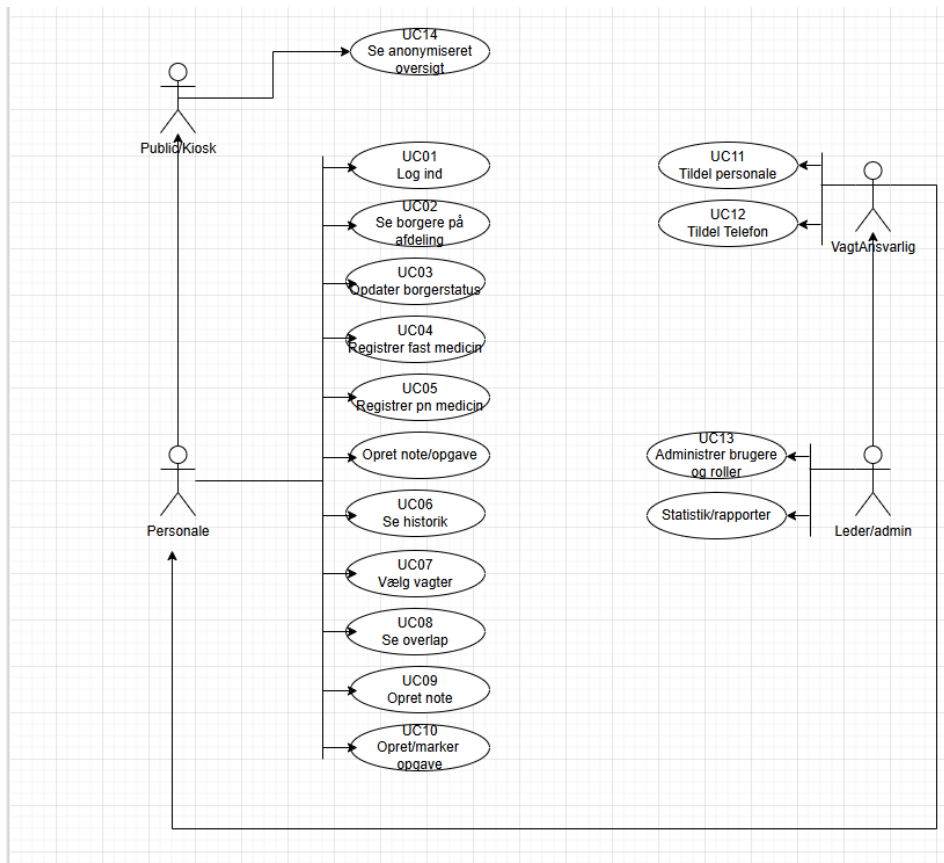
Dernæst har jeg prioriteret funktioner, **der** optimerer arbejdsgangen og de administrative opgaver, da de understøtter driften, men ikke er kritiske for patientsikkerhed eller lovkrav. På den måde sikrer jeg, at systemets kernefunktionalitet leverer reel værdi tidligt i projektet.

UC ID	Use Case	Primær aktør	Forretningsværdi	Risiko ved manglende løsning	Prioritet
UC01	Log ind (rollebaseret)	Personale	Høj	GDPR-brud	MUST
UC02	Se borgere på afdeling	Personale	Høj	Manglende overblik ved overlap	MUST
UC03	Opdatér borgerstatus / risikoprofil	Personale	Meget høj	Forkert risikovurdering	MUST
UC04	Registrér fast medicin	Personale	Meget høj	Medicinfejl / manglende dokumentation	MUST
UC05	Registrér PN medicin	Personale	Meget høj	Akut medicin uden dokumentation	MUST
UC06	Se historik / audit-log	Personale / Leder	Høj	Ingen sporbarhed	MUST
UC07	Vælg vagt	Personale	Høj	Ustruktureret vagtoverlap	SHOULD
UC08	Se overlap	Personale	Høj	Mangelfuld overlevering	SHOULD
UC09	Opret note	Personale	Mellem	Manglende kontekst	SHOULD
UC10	Opret opgave / markér udført	Personale	Mellem	Opgaver glemmes	SHOULD

UC ID	Use Case	Primær aktør	Forretningsværdi	Risiko ved manglende løsning	Prioritet
UC11	Tildel personale til borger	Vagtansvarlig	Mellem	Uklart ansvar	SHOULD
UC12	Tildel telefon til personale	Vagtansvarlig	Lav	Praktisk fordeling	COULD
UC13	Administrér brugere / roller	Leder	Mellem	Uklar adgangsstyring	SHOULD
UC14	Public anonym oversigtsskærm	Public	Mellem	GDPR-udfordring	SHOULD
UC15	Statistik / rapporter	Leder	Lav	Mindre styring	COULD

5.6 Use Case Diagram

Use case-diagrammet viser systemets centrale aktører (Personale, Vagtansvarlig, Leder/Admin og Public) samt deres tilhørende funktioner. Diagrammet illustrerer rollebaseret adgang og giver et samlet overblik over, hvordan overlapsystemet understøtter daglig drift, medicinhåndtering, vagtskifte og administration.



5.7 Brief Use Cases

UC01 – Log ind (rollebaseret)

Primær aktør: Bruger

Formål: At få adgang til systemet via login og blive sendt videre baseret på rolle.

Kort beskrivelse:

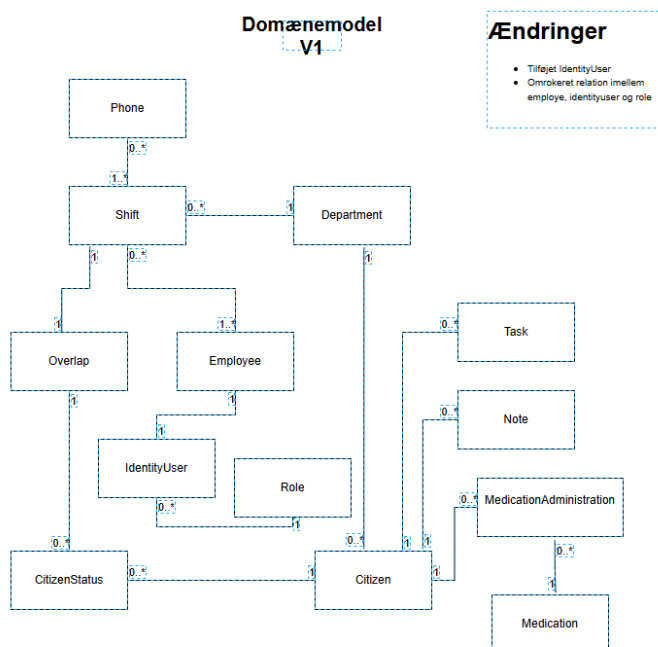
Brugeren åbner login-skærmen og indtaster sine loginoplysninger. Systemet validerer oplysningerne via Identity. Hvis login er korrekt, oprettes en session, og brugeren navigeres til næste skærm afhængigt af sin rolle. Hvis login fejler, vises en fejlbesked.

6 Domænemodel og Analyse

6.1 Domænemodel (Analysemodel)

Domænemodellen viser de centrale begreber i overlapsystemets domæne samt relationerne mellem dem. Modellen er konceptuel og indeholder udelukkende forretningsmæssige

begreber uden tekniske detaljer. Den illustrerer sammenhængen mellem borgere, vagter (Shift), overlap, statusregistreringer (CitizenStatus), medicinhandling samt medarbejdere og organisatorisk struktur. Formålet er at skabe en fælles forståelse af domænet før design og implementering.



6.2 Begrebsafklaring

6.3 Analyseovervejelser

7 Systemsekvens og Interaktionsanalyse

7.1 System Sequence Diagram – Medicin

7.2 System Sequence Diagram – Vagtskifte

7.3 Analyse af systemgrænse

Systemgrænsen for VIGOR kan beskrives som en klient–server arkitektur bestående af tre primære lag: klient, backend og database.

Brugerne interagerer med systemet gennem en .NET MAUI Blazor Hybrid-klient, som udgør systemets brugergrænseflade. Klienten håndterer præsentation, navigation og brugerinput, men indeholder ikke systemets centrale forretningslogik.

Når brugeren udfører handlinger i systemet, sendes forespørgsler fra klienten til et ASP.NET Core Web API. API'et fungerer som systemets backend og håndterer forretningslogik, autentificering, autorisation samt adgang til data. Autentificering sker via ASP.NET Identity, og adgang til beskyttede funktioner styres gennem rollebaseret autorisation.

Backend-systemet kommunikerer herefter med systemets database, hvor brugere, vagter og øvrige systemdata lagres og vedligeholdes.

Systemgrænsen kan derfor overordnet beskrives som følgende struktur:

MAUI-klient → Web API → Database

Denne opdeling sikrer en klar adskillelse mellem præsentationslag, applikationslogik og datalagring, hvilket øger systemets skalerbarhed, vedligeholdbarhed og mulighed for fremtidige udvidelser.

8 Design og Arkitektur

8.1 Overordnet arkitektur

VIGOR er designet med en lagdelt arkitektur, der adskiller præsentation, forretningslogik og dataadgang. Løsningen består af flere projekter, som hver har et klart ansvarsområde.

MAUI Blazor Hybrid-applikationen fungerer som klient og håndterer brugergrænseflade og interaktion. Klienten kommunikerer med systemets backend via HTTP-baserede API-kald.

Web-projektet fungerer som server og eksponerer REST-baserede API-endpoints, hvor forretningslogik og datahåndtering placeres. API'et fungerer som systemets centrale applikationslag og håndterer blandt andet brugeradministration, validering og adgang til systemets data.

Autentificering og autorisation håndteres af ASP.NET Identity, som fungerer som systemets identity provider. Når en bruger logger ind, udstedes et JWT (JSON Web Token), som klienten anvender i efterfølgende API-kald til at identificere brugeren og sikre rollebaseret adgang til systemets funktioner.

Shared-projektet indeholder fælles kontrakter, domænemodeller og DTO'er, som deles mellem klient og server, hvilket sikrer konsistens i dataudvekslingen mellem lagene.

Arkitekturen understøtter separation of concerns og reducerer koblingen mellem lagene. Præsentationslaget afhænger af abstractions (interfaces) fremfor konkrete implementeringer, hvilket gør systemet mere fleksibelt og testbart.

Strukturen forbereder systemet til fremtidig udvidelse, herunder mulig web-deployment eller integration med eksterne systemer.

8.2 Valg af teknologi (.NET MAUI Blazor Hybrid)

VIGOR er udviklet på Microsofts .NET-plattform og benytter en kombination af moderne teknologier, der understøtter både klientapplikation, backend og datahåndtering.

Klientapplikationen er udviklet som en .NET MAUI Blazor Hybrid-applikation. Denne teknologi gør det muligt at kombinere Blazor-komponenter med en native MAUI-applikation, hvilket giver mulighed for at udvikle en moderne brugergrænseflade baseret på webteknologier samtidig med, at applikationen kan køre som en lokal klient.

Backend-delen er implementeret som et ASP.NET Core Web API, som håndterer systemets forretningslogik og fungerer som bindeled mellem klienten og databasen. Kommunikation mellem klient og server foregår via HTTP-baserede API-kald.

Til dataadgang anvendes Entity Framework Core, som fungerer som ORM (Object Relational Mapper). Dette gør det muligt at arbejde med databasen gennem C#-objekter og samtidig holde dataadgangen adskilt fra den øvrige forretningslogik.

Brugerhåndtering og autentificering håndteres gennem ASP.NET Identity. Systemet anvender JWT (JSON Web Tokens) til autentificering mellem klient og server, hvor tokenet bruges til at identificere brugeren og styre rollebaseret adgang til systemets funktioner.

Valget af disse teknologier understøtter en moderne og skalerbar arkitektur, hvor komponenterne er tydeligt adskilt, samtidig med at de er tæt integreret i .NET-økosystemet.

8.3 MVC-struktur

8.4 Dependency Injection

Dependency Injection anvendes konsekvent i hele løsningen for at reducere kobling mellem komponenter og understøtte testbarhed. Services registreres centralt via `ServiceCollectionExtensions` i de respektive projekter, hvilket sikrer en ensartet og overskuelig konfiguration.

Ved at programmere mod interfaces fremfor konkrete implementeringer opnås fleksibilitet i udskiftning af komponenter, eksempelvis ved overgang fra in-memory repository til databasebaseret løsning.

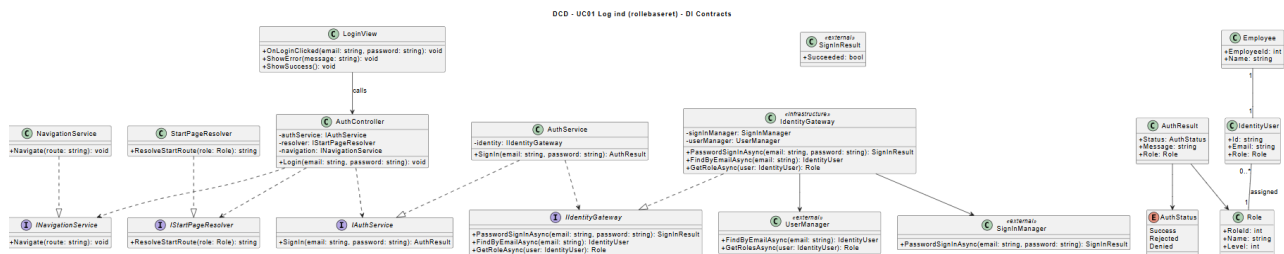
Denne tilgang understøtter SOLID-principperne, særligt Dependency Inversion Principle, og bidrager til en mere vedligeholdbar og skalerbar arkitektur.

8.5 SOLID, GRASP, KISS, DRY

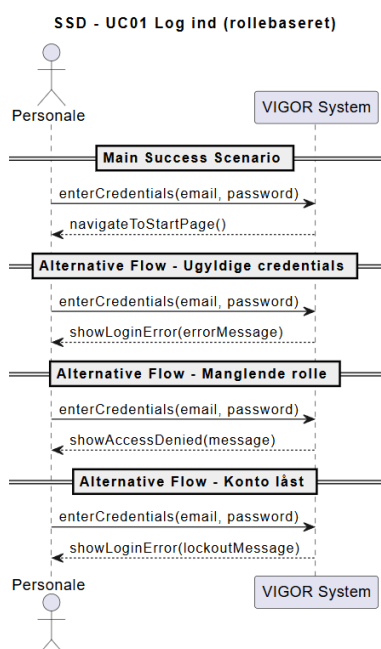
9 Designmodeller

9.1 Design Class Diagram (DCD)

Design class diagrammet anvendes til at beskrive systemets konkrete softwaredesign på klasseniveau. Diagrammet viser klasser, deres ansvar, relationer og afhængigheder. For UC01 illustrerer DCD'en opbygningen efter MVC-princippet samt brugen af interfaces og Dependency Injection for at reducere kobling. DCD'en danner grundlag for implementeringen og sikrer sporbarhed fra analyse til kode.

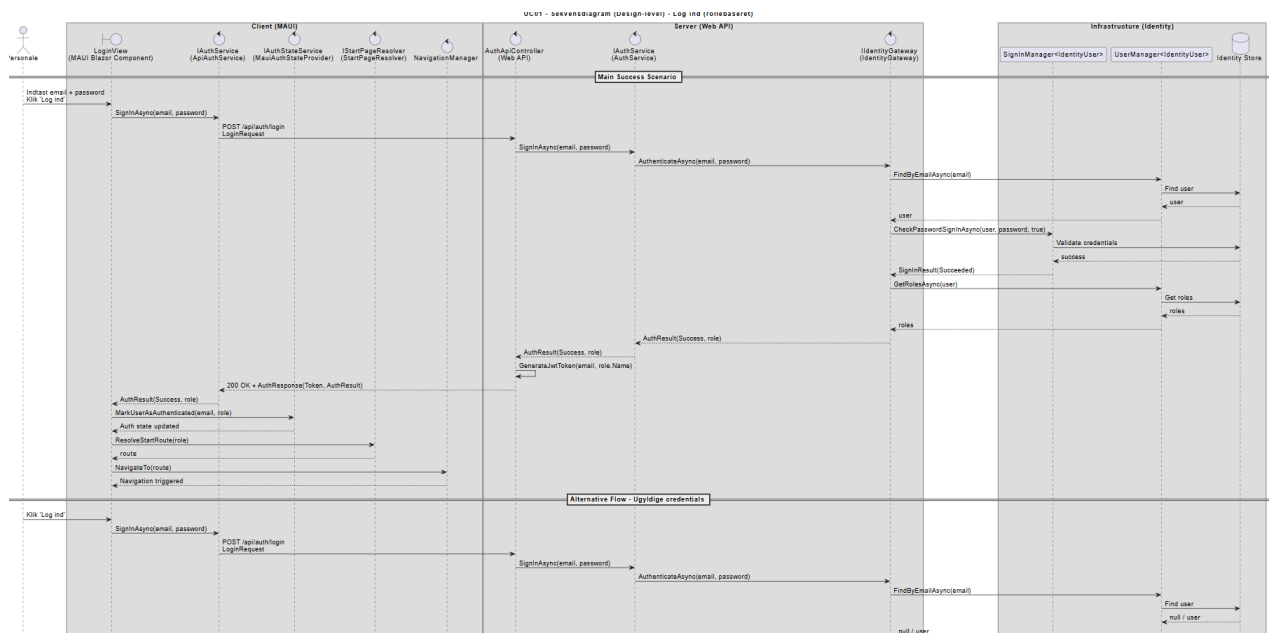


9.2 SSD – UC01 – Login



System Sequence Diagrammet beskriver interaktionen mellem aktøren og systemet. For UC01 viser SSD'en de systemkald, der udføres ved login, samt systemets respons ved succes, ugyldige credentials og manglende rolle. Diagrammet tydeliggør systemets ansvar og grænseflade. SSD'en danner grundlag for det efterfølgende sekvensdiagram.

9.3 Sekvensdiagram – UC01 – Login



p.s. Da sekvensdiagrammet er for stort til at vise i fuld størrelse her, så henvises til billaget for de fulde detaljer.

9.4 Ansvarsfordeling i klasser

10 Datamodel

10.1 ER-diagram

10.2 Entiteter og relationer

10.3 Historik- og audit-strategi

For at sikre sporbarhed og dokumentation er systemet designet med en append-only historikstrategi. Dette betyder, at ændringer ikke overskrives eller slettes, men i stedet gemmes som nye registreringer med tidsstempel og brugerreference.

Denne tilgang understøtter krav om dokumentation i forbindelse med medicinhåndtering og risikoprofilopdateringer. Den bidrager samtidig til GDPR-compliance ved at sikre transparens i forhold til, hvem der har foretaget ændringer og hvornår.

Append-only-princippet reducerer risikoen for manipulation af historiske data og danner grundlag for senere implementering af audit-log og eventuel rapportering.

11 Brugergænseflade

- 11.1 Designprincipper
 - 11.2 Wireframe – Public View
 - 11.3 Wireframe – Login
 - 11.4 Wireframe – Overlap screen
 - 11.5 Brugervenlighed for IT-ukyndige
-

12 Medicinhåndtering

- 12.1 Aktivitetsdiagram – Fast medicin
 - 12.2 PN medicin-flow
 - 12.3 Validering og logning
-

13 Ikke-funktionelle Krav (Supplementary Specification)

- 13.1 Brugervenlighed
- 13.2 Sikkerhed

Sikkerhed i VIGOR håndteres gennem en kombination af autentificering og autorisation. Systemet benytter ASP.NET Identity til håndtering af brugerkonti, loginoplysninger og rollebaseret adgangsstyring.

Når en bruger logger ind i systemet, valideres brugerens oplysninger af backend-systemet via ASP.NET Identity. Ved succesfuld autentificering udstedes et JSON Web Token (JWT), som anvendes til at identificere brugeren ved efterfølgende forespørgsler til systemets Web API.

JWT-tokenet sendes med i klientens API-kald og gør det muligt for serveren at verificere brugerens identitet uden at skulle gemme sessionstilstand på serveren. Dette gør autentificeringen både sikker og skalerbar.

Systemet anvender desuden rollebaseret adgangskontrol, hvor brugerens rolle bestemmer, hvilke funktioner og områder af systemet der er tilgængelige. Dette sikrer, at kun autoriseret personale kan tilgå bestemte funktioner i systemet.

Denne sikkerhedsmodel bidrager til at beskytte systemets data og sikre, at adgangen til funktioner og information styres korrekt.

13.3 Performance

13.4 Tilgængelighed

13.5 Vedligeholdbarhed

14 Risikovurdering og GDPR

14.1 GDPR-kontekst

14.2 Risikotabel

14.3 Anonymisering vs. lukket system

14.4 Rollebaseret adgang

14.5 Audit-log og sporbarhed

14.6 Argumentation for løsning

15 Deployment og Infrastruktur

VIGOR er designet som et internt system, der implementeres i et lukket miljø på bostedet. Løsningen består af en klientapplikation (.NET MAUI Blazor Hybrid) samt en serverkomponent (ASP.NET Core Web API).

MAUI-applikationen fungerer som systemets klient og installeres på de enheder, hvor personalet anvender systemet. Klienten håndterer brugergrænsefladen og kommunikerer med serveren via HTTP-baserede API-kald.

Serverkomponenten håndterer systemets forretningslogik, autentificering og datahåndtering. Web API'et kan hostes på en lokal server i bostedets interne netværk. Systemets database kan enten placeres på samme server som API'et eller hostes separat afhængigt af den ønskede driftsopsætning.

I første version forudsættes lokal installation og drift uden integration til eksterne systemer. Dette reducerer kompleksitet og sikkerhedsrisici i den indledende fase.

Systemet er arkitektonisk forberedt til senere udvidelse, herunder eventuel web-deployment eller integration med regionale systemer. Rollebaseret adgang og anonymiseret visning sikrer, at systemet kan anvendes både internt og i fællesområder uden at kompromittere persondata.

15.1 Deployment diagram

15.2 Lukket installationsscenario

15.3 Ekstern anonym adgang

15.4 Driftsovervejelser

16 Implementering

16.1 Projektstruktur

Løsningen er opdelt i flere projekter for at sikre klar ansvarsfordeling og lav kobling mellem komponenter.

VIGOR indeholder MAUI Blazor Hybrid-applikationen og håndterer brugergrænseflade og klientlogik.

VIGOR.Web indeholder serverlogik, API-endpoints og dataadgang.

VIGOR.Shared indeholder fælles kontrakter, domænemodeller og DTO'er, som deles mellem klient og server.

Denne struktur understøtter lagdeling og separation of concerns og gør det muligt at udvikle og teste komponenter isoleret. Projektopdelingen bidrager til vedligeholdbarhed og giver fleksibilitet i forhold til fremtidige ændringer eller udvidelser.

16.2 Mappestruktur og namespaces

16.3 Centrale services

16.4 Teststrategi

I dette projekt anvendes både automatiserede tests og manuelle tests til at verificere systemets funktionalitet. Formålet med teststrategien er at sikre, at centrale dele af systemets forretningslogik fungerer korrekt og stabilt gennem hele udviklingsprocessen.

I iteration 1 har fokus været på at teste de komponenter, der håndterer autentifikation og rollebaseret navigation. Disse dele er centrale for systemets adgangskontrol og brugeroplevelse, da de afgør, om en bruger kan logge ind, samt hvilken startside brugeren bliver sendt til efter login.

Teststrategien for iteration 1 har derfor haft fokus på:

- Verifikation af login-flowet
- Verifikation af rollebaseret navigation
- Kontrol af fejlscenarier ved login

Automatiserede unit tests er anvendt til at teste systemets centrale beslutningslogik isoleret fra eksterne afhængigheder. Eksterne komponenter, såsom autentifikationsgatewayen, er mock'et i tests, så kun den relevante forretningslogik bliver testet.

Derudover er der gennemført manuelle tests af systemets brugergrænseflade for at verificere, at login-processen fungerer korrekt fra brugerens perspektiv.

Denne kombination af automatiserede tests og manuelle tests bidrager til at sikre systemets stabilitet og korrekt funktion i de tidlige iterationer.

16.5 Eksempler på kode

17 Test og Kvalitet

17.1 Unit tests

Der er implementeret automatiserede unit tests for centrale komponenter i systemets login- og navigationsfunktionalitet. Unit tests er anvendt til at teste systemets forretningslogik isoleret fra eksterne systemer.

Testene er implementeret i et separat testprojekt ved hjælp af xUnit som testframework og Moq til mocking af afhængigheder.

Følgende komponenter er testet:

17.1.1 StartPageResolver

StartPageResolver er ansvarlig for at bestemme hvilken startside en bruger skal sendes til efter login baseret på brugerens rolle.

Følgende scenarier er testet:

- Leder navigeres til /admin
- Vagtansvarlig navigeres til /schedule
- Personale navigeres til /home
- Ukendt rolle navigeres til standardstartside
- Manglende rolle navigeres til standardstartside

Disse tests sikrer, at systemets rollebaserede navigation fungerer korrekt.

17.1.2 AuthService

AuthService indeholder systemets centrale loginlogik og håndterer autentifikation via en gateway til Identity-systemet.

Følgende scenarier er testet:

- Succesfuldt login med gyldige credentials
- Login med ugyldige credentials
- Login hvor brugeren mangler en gyldig rolle
- Login hvor brugeren er låst efter for mange mislykkede loginforsøg

Eksterne afhængigheder til autentifikationssystemet er mock'et, så testen kun verificerer AuthService's egen logik.

17.1.3 AuthApiController

AuthApiController håndterer API-endpointet for login og returnerer HTTP-respons til klienten.

Følgende scenarier er testet:

- Login med gyldige credentials returnerer HTTP 200 OK
- Login med ugyldige credentials returnerer HTTP 401 Unauthorized
- Login hvor brugeren mangler rolle returnerer HTTP 401 Unauthorized

Disse tests verificerer, at controlleren returnerer de korrekte HTTP-statuskoder og svarstrukturer.

Samlet er der implementeret 12 unit tests, som dækker de vigtigste beslutningspunkter i systemets login-flow.

17.2 Integrationstest

17.3 Manuel test

Ud over de automatiserede tests er der gennemført manuelle tests af login-funktionaliteten via systemets brugergrænseflade.

Formålet med de manuelle tests har været at verificere, at login-processen fungerer korrekt fra brugerens perspektiv, og at navigationen efter login sker som forventet.

Følgende scenarier er manuelt testet:

- Login med gyldige brugeroplysninger
- Login med forkert adgangskode
- Login med bruger uden korrekt rolle
- Login med låst konto

Ved succesfuldt login verificeres det, at brugeren bliver sendt til den korrekte startside baseret på brugerens rolle.

Manuel test anvendes som supplement til de automatiserede tests for at sikre, at systemets samlede login-flow fungerer korrekt i praksis.

17.4 Kvalitetssikring

Kvalitetssikring i projektet sker gennem en kombination af kodeprincipper, automatiserede tests og løbende verifikation af systemets funktionalitet.

Udviklingen følger etablerede principper som SOLID, DRY, KISS og GRASP, hvilket bidrager til en klar ansvarsfordeling mellem systemets komponenter og en mere testbar arkitektur.

Automatiserede unit tests anvendes til at verificere systemets centrale beslutningslogik. Testene udføres løbende under udviklingen og sikrer, at ændringer i koden ikke introducerer fejl i eksisterende funktionalitet.

Derudover gennemføres manuelle tests af centrale funktioner i systemets brugergrænseflade for at verificere systemets samlede funktionalitet fra brugerens perspektiv.

Kombinationen af arkitekturelle principper, automatiserede tests og manuel verifikation bidrager til en stabil og robust implementering af systemets funktionalitet.

18 Refleksion

- 18.1 Metoderefleksion
 - 18.2 Samarbejde og iterationer
 - 18.3 Hvad fungerede godt
 - 18.4 Hvad kunne forbedres
-

19 Konklusion

- 19.1 Opfyldelse af problemformulering
 - 19.2 Fremtidige udvidelser
-

20 Referencer

21 Bilag

21.1 Ekstra diagrammer

21.2 Ekstra use cases

UC01 – Log ind (rollebaseret)

Scope: VIGOR(MAUI Blazor Hybrid)

Primær aktør: Personale

Sekundære aktører: VagtAnsvarlig, Leder/Admin

Interessenter og interesser:

- **Bruger** (Personale/VagtAnsvarlig/Leder/Admin)
 - Ønsker hurtig og sikker adgang til systemet og kun adgang til funktioner, der matcher deres rolle.
- **Bostedet Slottet**
 - Ønsker korrekt adgangskontrol og mulighed for sporbarhed (hvem har været logget ind).
- **Staten / Tilsynsmyndigheder (fx Datatilsynet)**
 - Har interesse i, at systemet overholder gældende lovgivning, herunder GDPR og regler om behandling af personoplysninger i sundheds- og omsorgssektoren.

Systemet: Skal beskytte mod uautoriseret adgang og håndtere fejl robust.

Preconditions:

1. Brugeren har en oprettet konto i Identity.
2. Brugeren er tildelt mindst én rolle.
3. Systemet (app og autentificering) er tilgængeligt.

Minimal Guarantees (ved fejl)

- Ingen session oprettes.
- Brugeren får en forståelig fejlbesked.
- Loginforsøg kan logges til audit/sporbarhed.

Success Guarantees (ved succes)

- Brugeren er autentificeret.
- Brugers rolle(r) er indlæst.
- Brugeren navigeres til korrekt startside baseret på rolle.

Trigger: Brugeren åbner appen og bliver præsenteret for login-skærmen.

Main Success Scenario:

1. Brugeren ser login-skærmen.
2. Brugeren indtaster brugernavn/email og adgangskode.
3. Brugeren vælger "Log ind".
4. Systemet validerer loginoplysninger via Identity.
5. Systemet opretter en autentificeret session.
6. Systemet indlæser brugerens rolle(r).
7. Systemet autoriserer adgang baseret på rolle.
8. Systemet navigerer brugeren til relevant startskærm.

Alternative flows:

E1 – Ugyldige loginoplysninger

- 4a. Identity afviser credentials.
- 4b. Systemet viser fejlbesked: "Forkert brugernavn eller adgangskode."
- 4c. Brugeren forbliver på login-skærmen.

E2 – Konto låst

- 4a. Identity returnerer "locked out".
- 4b. Systemet informerer brugeren om, at kontoen er midlertidigt låst.
- 4c. Brugeren kan ikke logge ind før logout ophører eller admin ændrer status.

E3 – Manglende rolle/adgang

- 6a. Systemet finder ingen gyldig rolle.
- 6b. Systemet afviser adgang og viser besked: "Du har ikke adgang. Kontakt administrator."
- 6c. Ingen adgang gives til systemets funktioner.

E4 – Systemfejl

- 4a. Autentificering fejler på grund af teknisk fejl.
- 4b. Systemet viser generel fejlbesked.
- 4c. Fejlen logges teknisk.

Frekvens: Dagligt, typisk ved start af vagt eller ved genåbning af appen.

21.3 Screenshots

21.4 GitHub link